WORKSHOP SESSION

Multi-Agent Configuration

with Config Control for Kiro

Building governed, scalable AI agent systems through configuration

🎯 orchestrator · 🔍 readonly · ✍️ write-agent · 🎮 game-builder

200 Level · 45 Minutes · No AWS Account Required

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The Problem

01

> *AI agents are powerful. But without guardrails, one bad prompt can delete your codebase.*

- Agents have unlimited tool access by default
- No way to restrict capabilities per task or per agent
- No delegation — one agent does everything
- No audit trail — no warnings, no blocks, no logs
- No way to share agent config across teams

In regulated environments — finance, healthcare, government — this is a non-starter.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What if you could...

02

- ✅ **Configure** agent behavior visually — tools, permissions, guardrails
- ✅ **Govern** with hooks that warn or block dangerous actions at runtime
- ✅ **Delegate** to specialized subagents with least-privilege tool access
- ✅ **Define tasks** as swappable skill specs — same agent, different output
- ✅ **Share** the entire setup as a folder — `git commit`, anyone gets the same system

> *All of this exists today. It's a folder called `.kiro/`*

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The Solution: .kiro/

03

```
.kiro/
├── agents/
│   ├── orchestrator.json
│   ├── readonly.json
│   ├── write-agent.json
│   └── game-builder.json
├── settings/
│   └── mcp.json
├── steering/
│   └── orchestrator-rules.md
└── skills/
    ├── safe-research/
    └── kiropong/
```

This folder IS your agent system

Version controlled

Commit to git, review in PRs

Shareable

Clone the repo → same agent setup

Auditable

Every rule is readable plain text

Visual

Edit in cckiro or JSON — they stay in sync

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The Architecture

One orchestrator, three specialists — least privilege by design

🎯

Orchestrator

All 9 tools · Governed by steering + hooks · Delegates to subagents

↓ delegates to ↓

🔍

readonly

`read` + `code`

Cannot modify anything

✍️

write-agent

`read` + `write` + `shell`

Asks before every change

🎮

game-builder

`read` + `write` + `shell`

Skill defines the output

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

5 Layers of Governance

04

Defense in depth — each layer catches what the previous one missed

#	Layer	What it does	Example
1	Tools	What the agent CAN do	readonly: only `read` + `code`
2	Auto-Allow	What runs WITHOUT permission	`read` auto-allowed, `write` needs approval
3	Tool Settings	Per-tool restrictions	`shell` deniedCommands: `rm`, `mv`, `chmod`
4	Hooks	Runtime warnings or blocks	preToolUse on `write` → warns or blocks
5	Steering	Behavioral rules in English	"Write tasks → delegate to write-agent"

> *Change `exit 0` to `exit 2` in a hook → the agent is blocked, not just warned. One number changes the behavior.*

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Tools & Permissions

Orchestrator: all 9 tools

`read`	Read files	auto
`glob`	Find files	auto
`grep`	Search contents	auto
`code`	Code intelligence	auto
`write`	Create/modify	approval
`shell`	Run commands	approval
`web_search`	Search web	approval
`web_fetch`	Fetch pages	approval
`use_subagent`	Delegate	approval

Tool aliases

`@aws-knowledge/search_documentation` → `kb_search`

Denied commands

`rm *`, `mv *`, `cp *`, `chmod *`

Subagent config

Orchestrator can delegate to: `readonly`, `write-agent`, `game-builder`

> The orchestrator has ALL tools but is governed. It CAN write — but hooks warn it, and steering tells it to delegate instead.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Hooks — Runtime Guardrails

Hook	When	Command	Exit
agentSpawn	Agent starts	`echo '🎯 Orchestrator active'`	0
preToolUse write	Before write	`echo '⚠️ Consider delegating'`	0
preToolUse shell	Before shell	`echo '⚠️ Consider delegating'`	0
stop	Agent finishes	`echo '✅ Task complete'`	0

`exit 0` — Allow

Action proceeds. Warning logged. Agent may self-correct.

`exit 2` — Block

Action stopped. Agent must find another way — like delegating.

Matchers target specific tools. Without a matcher, the hook fires for every tool call.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Steering — The Brain

```
# Orchestrator Rules

## When to delegate
- Write tasks → write-agent
- Investigation → readonly
- Game building → game-builder

## When to act directly
- Simple reads, quick answers
- MCP documentation lookups

## Always
- Explain what you're doing
- Report back after delegation
```

Why steering matters

- Rules in **plain English**
- Loaded via `file://` reference
- Version controlled in git
- Change rules → behavior changes immediately
- No code changes needed

> *Your compliance team can read and approve steering rules. Try doing that with a neural network.*

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Skills — Swappable Task Specs

The game-builder agent is **generic**. Its prompt says: *"follow the skill instructions exactly."*

🏓

kiropong/SKILL.md

600×300 canvas · paddle physics · vs AI / 2 Player · First to 5 / Timed 45s

👾

space-invaders/SKILL.md

Same agent → different game

How skills work

- **Progressive loading** — metadata at startup, full content on demand
- **Keyword triggers** — "compare", "vs" activates safe-research
- **Reference files** — supporting docs the agent reads during workflow

The pattern

Same agent + different skill = different output.
Swap the skill, don't rebuild the agent.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

MCP Servers

Model Context Protocol connects agents to external data sources and APIs.

aws-knowledge

URL: `https://knowledge-mcp.global.api.aws`

6 tools: search, read, recommend, regions

Three configuration levels

Agent	On the agent JSON — portable
Workspace	`.kiro/settings/mcp.json`
Global	`~/.kiro/settings/mcp.json`

Per-agent assignment

Agent	MCP
orchestrator	✅ aws-knowledge
write-agent	✅ aws-knowledge
readonly	❌ not needed
game-builder	❌ not needed

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Resources — Agent Context

Type	URI	Loading
File	`file://`	Full at startup
Skill	`skill://`	Metadata at startup, full on demand

Per-agent assignment

Agent	Resources
orchestrator	Steering + safe-research skill
game-builder	kiropong skill
readonly	None
write-agent	None

Compatible files

`.md` `.txt` `.py` `.js` `.ts` `.json` `.yaml`

> *Without resources, the agent only knows what's in its prompt. With resources, it loads rules and specs automatically.*

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Subagents — Least Privilege

🔍

readonly

Tools: read, code

Can: Read, analyze, explain

Cannot: Write, delete, run commands

Zero risk of modification

✍️

write-agent

Tools: read, write, shell

Can: Create and modify files

Constraint: Approval before every write

Human-in-the-loop

🎮

game-builder

Tools: read, write, shell

Can: Build from skill specs

Constraint: Generic — skill defines output

Reusable across tasks

> *The orchestrator has all tools but is governed. Subagents have limited tools by design. This is least privilege applied to AI agents.*

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Config Control for Kiro

Visual management for everything in .kiro/

Dashboard
Overview of agents, MCP, steering, skills

Agents
Visual editor + JSON sync

MCP
Add, test, toggle servers

Steering
Create and edit rules

Skills
Browse and edit skill specs

Workspaces
Global vs project config

Runs locally at localhost:3030 · Single binary · No dependencies · Open source

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

For Regulated Environments

Requirement	How .kiro/ addresses it
Least privilege	Each agent has only the tools it needs
Audit trail	Hooks log every tool use with timestamps
Human-in-the-loop	Auto-allow controls which tools need approval
Blocked actions	Hooks with `exit 2` prevent dangerous operations
Denied commands	Tool settings block specific shell commands
Reviewable rules	Steering files are plain markdown
Version control	`.kiro/` commits to git like any code
Reproducible	Same folder = same agent system, every time
Separation of duties	Orchestrator delegates, subagents execute

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Key Takeaways

🎯 **Agents are config, not code.**

A .kiro/ folder IS your agent system. Share it, version it, review it.

🛡️ **Governance is built in.**

5 layers: tools, auto-allow, tool settings, hooks, steering.

🎮 **Skills make agents reusable.**

Same agent + different skill = different output.

📋 **Steering is the control plane.**

Plain English rules your compliance team can read and approve.

📁 **Configuration is the product.**

Git commit your .kiro/ folder. Anyone who clones gets the same system.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Try It Yourself

45 minutes · No AWS account · All local

Workshop

Multi-Agent Configuration
with Config Control for Kiro

[QR CODE]

Import config | Explore features | Test 24 behaviors | Build KiroPong

Thank you!

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.